

Classification

MATH 4432 Statistical Machine Learning
Tutorial T05

Class probabilities, decision boundaries and evaluation

Department of Mathematics, HKUST

Fall 2026

The classification problem

Predict a categorical response from an input vector.

$$\mathcal{D} = \{(x_i, y_i)\}_{i=1}^n, \quad x_i \in \mathbb{R}^p, \quad y_i \in \{1, \dots, C\}$$

Symbol	Meaning
n	Number of training observations
p	Number of input features
C	Number of classes
k	Number of neighbors in KNN

Augmented input for regression scores:

$$\tilde{x} = (1, x_1, \dots, x_p)^T$$

From class probabilities to the best decision

For an input x , describe uncertainty by the posterior class probabilities:

$$p_c(x) = P(Y = c \mid X = x), \quad \sum_{c=1}^C p_c(x) = 1$$

Under 0-1 loss, a wrong prediction costs 1; a correct prediction costs 0.

$$R(a \mid x) = P(Y \neq a \mid X = x) = 1 - p_a(x)$$

So the class with the largest probability has the smallest expected loss:

$$a^*(x) = \arg \max_c p_c(x)$$

The true probabilities are unknown. How can training data help us estimate them?

KNN estimates probabilities from nearby labels

K-nearest neighbors keeps the training examples.

For a new input, find its k closest training points and count their labels.

$$\hat{p}_c(x) = \frac{1}{k} \sum_{i \in N_k(x)} \mathbb{I}(y_i = c)$$

Example: 5 neighbors contain 1, 3 and 1 observations from the three classes.

$$\hat{p}(x) = (0.2, 0.6, 0.2), \quad \hat{y}(x) = 2$$

KNN is nonparametric: prediction uses the stored sample. k and the distance are tuning choices.

Why KNN uses the local class proportions

Treat the k nearby labels as a sample with common class probabilities q .

$$n_c(x) = \sum_{i \in N_k(x)} \mathbf{1}(y_i = c), \quad \sum_c n_c(x) = k$$

Maximize the local categorical log-likelihood, subject to probabilities summing to 1:

$$\ell(q) = \sum_c n_c \log q_c, \quad q_c \geq 0, \quad \sum_c q_c = 1$$

$$\frac{\partial}{\partial q_c} \left[\ell(q) + \lambda \left(1 - \sum_r q_r \right) \right] = \frac{n_c}{q_c} - \lambda = 0$$

$$\hat{q}_c = \frac{n_c}{\lambda}, \quad \sum_c \hat{q}_c = \frac{k}{\lambda} = 1 \quad \implies \quad \hat{p}_c(x) = \frac{n_c(x)}{k}$$

This is a local approximation: nearby inputs are assumed to have similar class probabilities.
If a class has no nearby observations, its maximum-likelihood estimate is 0.

Distance determines which observations are nearby

A positive definite matrix M defines a distance between inputs.

$$d_M(x, x') = \sqrt{(x - x')^T M (x - x')}$$

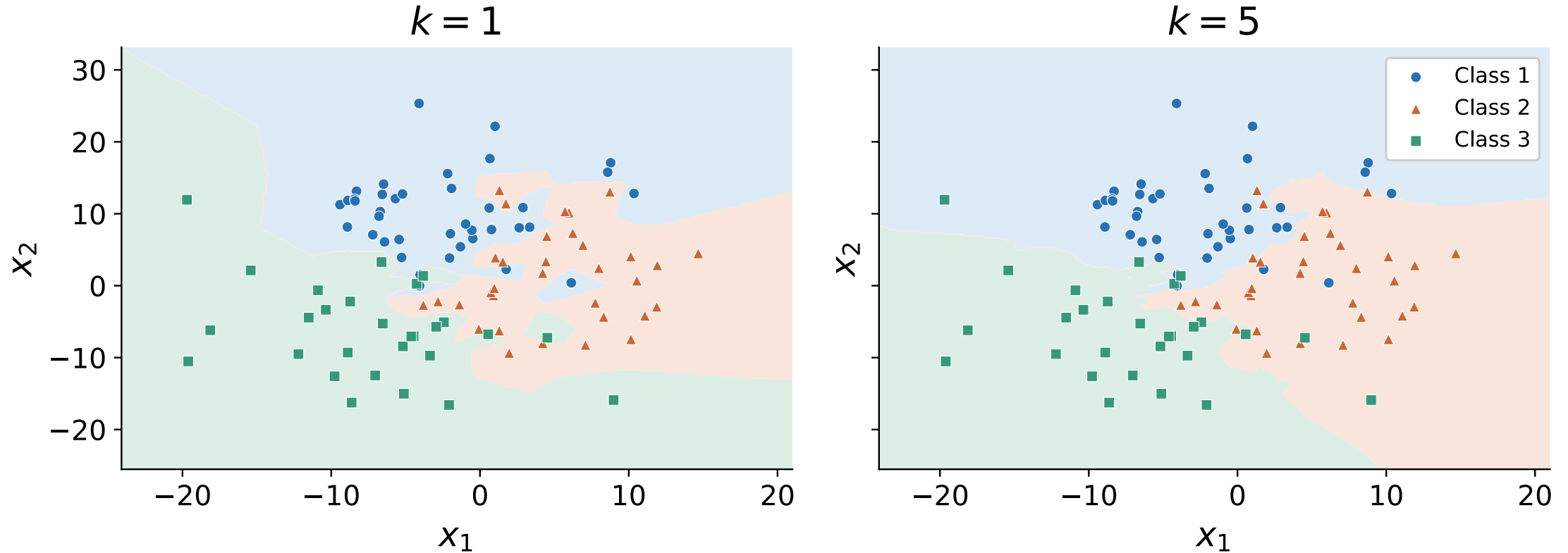
Euclidean distance uses $M = I$. Feature units then affect the neighbors.

Standardization puts each feature on its training-sample scale.

$$z_j = \frac{x_j - \bar{x}_{j, \text{train}}}{s_{j, \text{train}}}$$

Estimate scaling on training data only. Apply the same transformation to validation and test inputs.

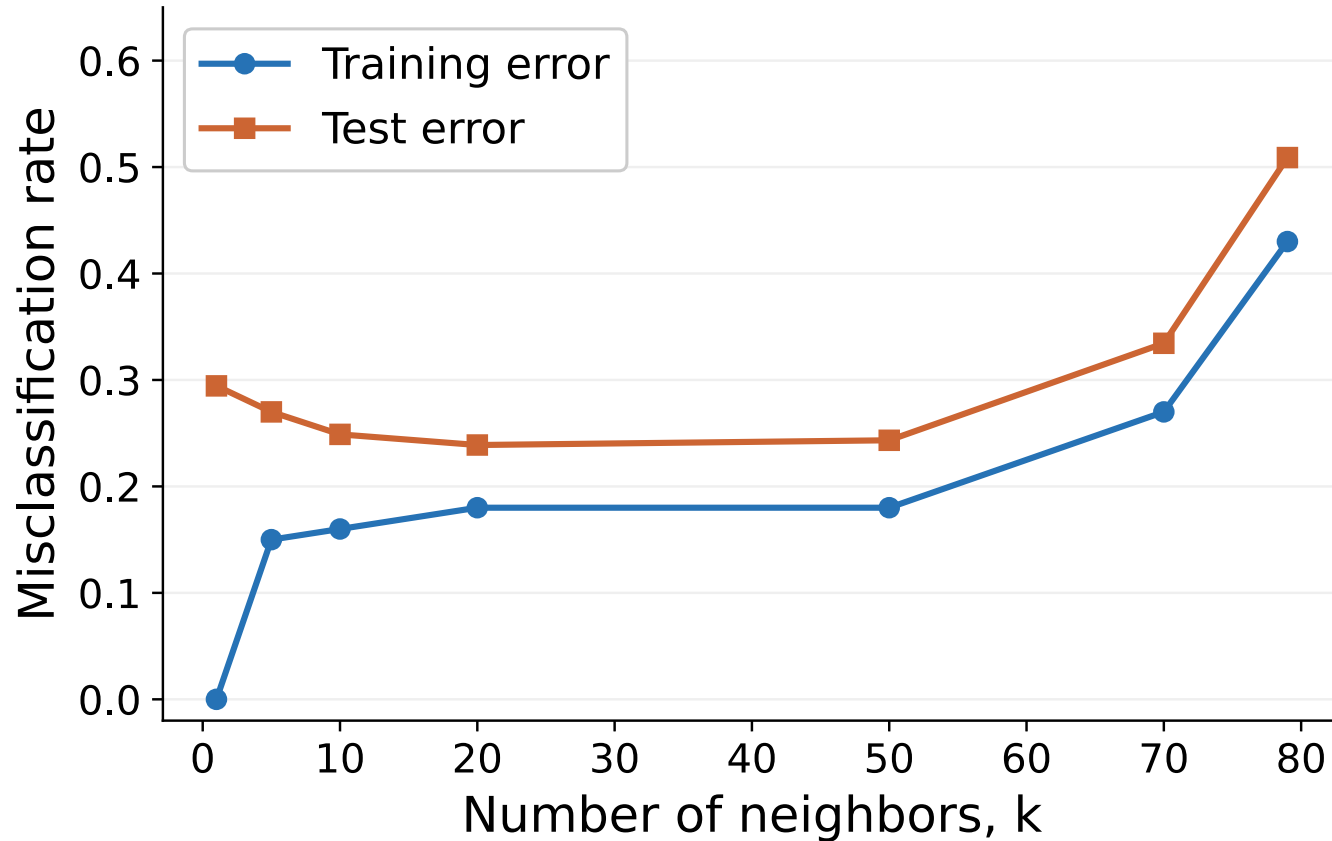
The number of neighbors changes the boundary



Small k follows local detail. Larger k averages over a wider neighborhood.

Same synthetic data in both panels: 100 training observations, three classes.

Choosing k requires data beyond the fitted sample



k = 1

Every training point is its own nearest neighbor.

Training error can be zero while test error remains substantial.

Use validation or cross-validation to select k.

The displayed test curve illustrates the trade-off. Keep a final test set untouched during tuning.

A shared model for class probabilities

KNN uses a new neighborhood for each prediction.
Sparse neighborhoods can make probability estimates unstable.

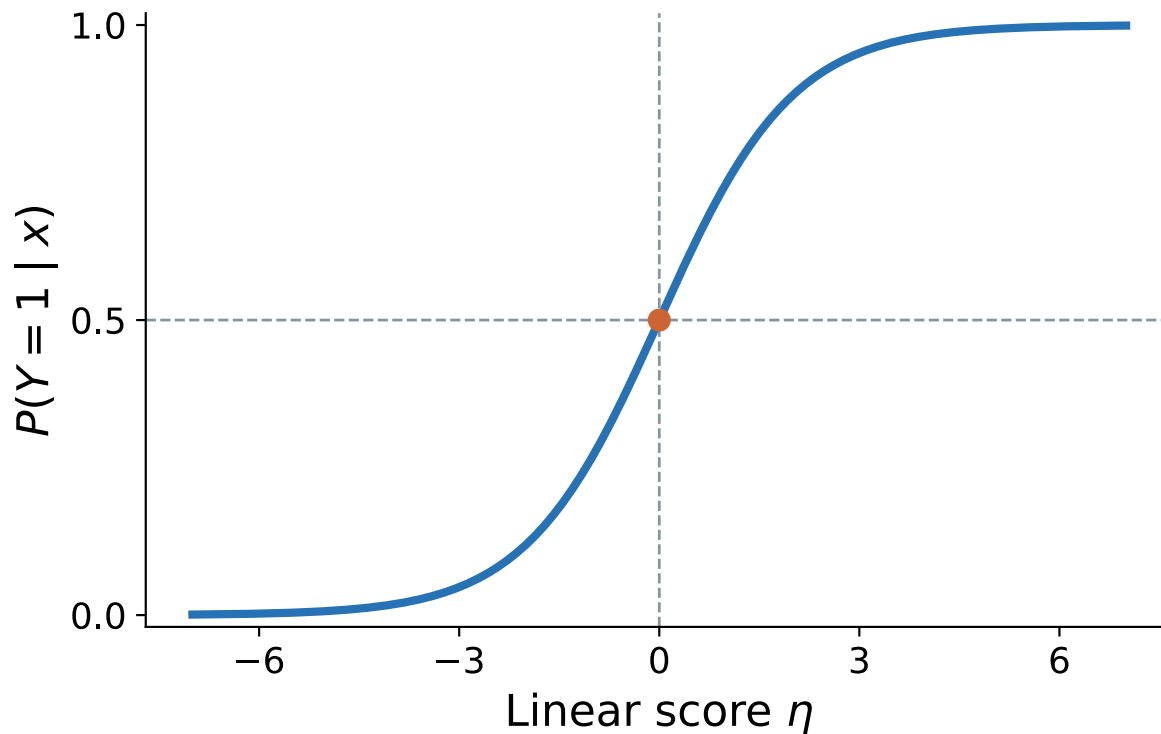
Could all observations help estimate one shared probability function?

A compact choice is a linear model for log odds:

$$\log \frac{p(x)}{1 - p(x)} = \beta^T \tilde{x}$$

Here $p(x) = P(Y = 1 \mid X = x)$. This assumption leads to logistic regression. Its value depends on whether the structure fits the data.

Linear log odds lead to the sigmoid



For two classes, write

$$\eta = \beta^T \tilde{x}, \quad p = P(Y = 1 \mid x)$$

$$\log \frac{p}{1-p} = \eta \implies \frac{p}{1-p} = e^\eta$$

$$p = e^\eta (1 - p)$$

$$p = \sigma(\eta) = \frac{e^\eta}{1 + e^\eta} = \frac{1}{1 + e^{-\eta}}$$

$$P(Y = y \mid x) = p^y (1 - p)^{1-y}$$

The sigmoid maps any real score into (0, 1). The last line is the Bernoulli model for $y = 0$ or 1 .

Coefficients describe changes in log odds

$$\text{logit}(p) = \log \frac{p}{1-p} = \beta_0 + \beta_1 x_1 + \cdots + \beta_p x_p$$

Holding the other features fixed, increasing x_j by 1 adds β_j to the log odds.

$$\frac{\text{odds}(x_j + 1)}{\text{odds}(x_j)} = e^{\beta_j}$$

If $\beta_j = \log 2$, the odds double. A probability of 0.20 becomes 1/3.

The change in probability depends on its starting value. The odds multiplier is constant.

Fitting logistic regression: the log-likelihood

Condition on the inputs and assume independent labels.

$$L(\beta) = \prod_{i=1}^n p_i^{y_i} (1 - p_i)^{1 - y_i}$$

$$\ell(\beta) = \sum_{i=1}^n [y_i \log p_i + (1 - y_i) \log(1 - p_i)]$$

Substitute: $\log p_i = \eta_i - \log(1 + e^{\eta_i}), \quad \log(1 - p_i) = -\log(1 + e^{\eta_i})$

$$\ell(\beta) = \sum_{i=1}^n [y_i \beta^T \tilde{x}_i - \log(1 + e^{\beta^T \tilde{x}_i})]$$

Minimizing $-\ell$ gives cross-entropy. For a single case, it is $-\log$ of the probability assigned to the observed label.

Gradient, Hessian and convexity

Let X contain the augmented inputs as rows, and p the fitted probabilities.

$$\frac{dp_i}{d\eta_i} = p_i(1 - p_i), \quad W = \text{diag} \{p_i(1 - p_i)\}$$

$$\nabla \ell = X^T(y - p), \quad \nabla^2 \ell = -X^T W X$$

The gradient gives the slopes; the Hessian describes curvature.

For any vector v :

$$v^T X^T W X v = \sum_i p_i(1 - p_i)(\tilde{x}_i^T v)^2 \geq 0$$

The negative log-likelihood is convex. A finite optimum still requires suitable rank and overlap conditions.

Newton updates and weighted least squares

Newton approximates the gradient and sets this approximation to zero.

$$0 \approx \nabla \ell(\beta) + \nabla^2 \ell(\beta)(\beta_{\text{new}} - \beta)$$

$$\beta_{\text{new}} = \beta + (X^T W X)^{-1} X^T (y - p)$$

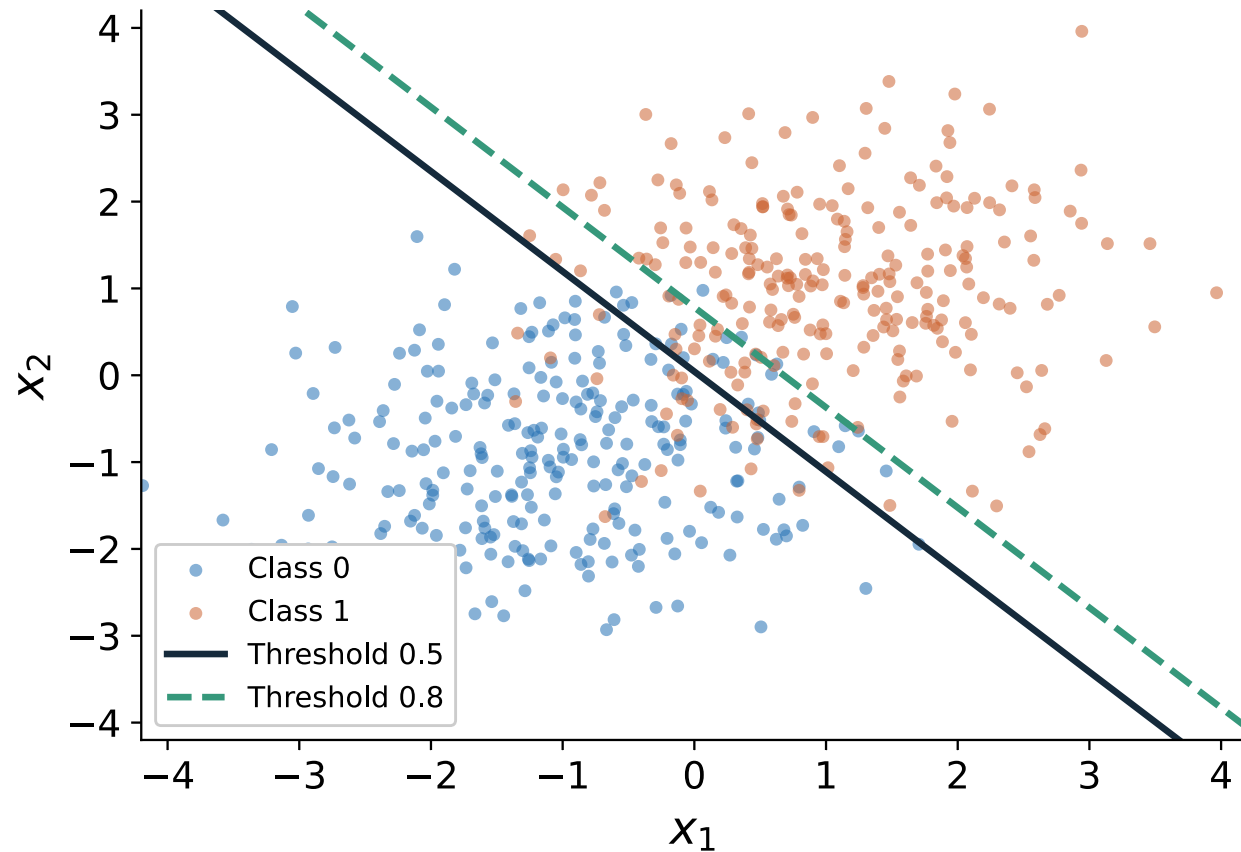
Define a working response, using the current probabilities and weights:

$$z = X\beta + W^{-1}(y - p)$$

$$\beta_{\text{new}} = (X^T W X)^{-1} X^T W z$$

This is iteratively reweighted least squares (IRLS). Recompute p and W after each update.

A probability threshold gives a decision boundary



Predict class 1 when $p(x)$ exceeds a threshold t .

$$\beta^T \tilde{x} = \log \frac{t}{1-t}$$

At $t = 0.5$, the boundary has linear score zero.

Changing t moves the boundary without refitting the coefficients.

Synthetic two-feature example. A linear score gives a straight boundary in these feature coordinates.

The same log-odds idea extends to multiple classes

A sigmoid describes two classes. With C classes, the probabilities must remain nonnegative and sum to one.

We extend the same linear log-odds idea relative to a baseline class C :

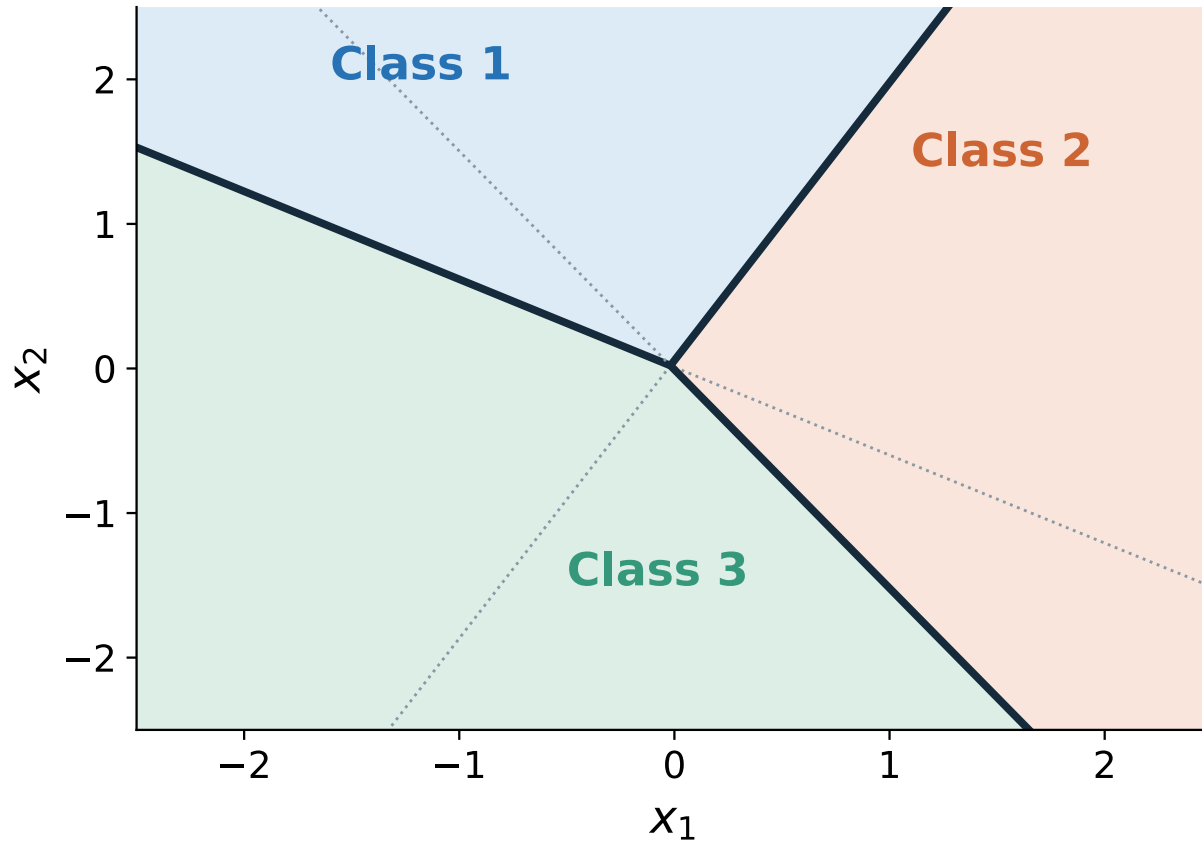
$$\log \frac{p_c(x)}{p_C(x)} = \eta_c = \beta_c^T \tilde{x} \quad (c < C), \quad \eta_C = 0$$

Solving these ratios together with the sum-to-one constraint gives softmax:

$$p_c(x) = \frac{e^{\eta_c}}{\sum_{r=1}^C e^{\eta_r}}, \quad c = 1, \dots, C$$

For $C = 2$, softmax reduces to the sigmoid. The baseline removes redundant parameters without changing the probability representation.

Pairwise log odds determine multiclass boundaries



Three fitted scores:

$$\eta_1 = -0.03 + 2.08x_1 + 3.42x_2$$

$$\eta_2 = 0.02 + 3.81x_1 + 2.52x_2$$

$$\eta_3 = 0$$

$$\log(p_a/p_b) = \eta_a - \eta_b$$

A decision boundary also requires the tied classes to beat every other class.

Solid segments separate predicted classes. Dotted extensions show pairwise equalities outside active boundaries.

A distributional explanation for linear log odds

Logistic regression assumes linear log odds.

Could a model for the inputs within each class produce that form?

$$\pi_c = P(Y = c), \quad f_c(x) = f(x \mid Y = c)$$

Bayes' rule expresses binary log odds in terms of class densities:

$$\log \frac{p(x)}{1 - p(x)} = \log \frac{\pi_1}{\pi_0} + \log \frac{f_1(x)}{f_0(x)}$$

What if each class is Gaussian, with a different mean but one shared covariance matrix?

This assumption defines LDA. Expanding the density ratio will show why its log odds are linear.

Gaussian densities give discriminant scores

In p input dimensions, assume a Gaussian distribution within each class.

$$f_c(x) = \frac{1}{(2\pi)^{p/2} |\Sigma_c|^{1/2}} \exp \left[-\frac{1}{2} (x - \mu_c)^T \Sigma_c^{-1} (x - \mu_c) \right]$$

Bayes classification maximizes $\pi_c f_c(x)$, or equivalently its logarithm.

$$\begin{aligned} \log[\pi_c f_c(x)] = & \log \pi_c - \frac{p}{2} \log(2\pi) - \frac{1}{2} \log |\Sigma_c| \\ & - \frac{1}{2} (x - \mu_c)^T \Sigma_c^{-1} (x - \mu_c) \end{aligned}$$

Terms common to every class can be removed without changing which class has the largest score.

Linear discriminant analysis (LDA)

Set every class covariance to the same matrix Σ and expand:

$$\begin{aligned} & -\frac{1}{2}(x - \mu_c)^T \Sigma^{-1} (x - \mu_c) \\ &= -\frac{1}{2}x^T \Sigma^{-1} x + x^T \Sigma^{-1} \mu_c - \frac{1}{2}\mu_c^T \Sigma^{-1} \mu_c \end{aligned}$$

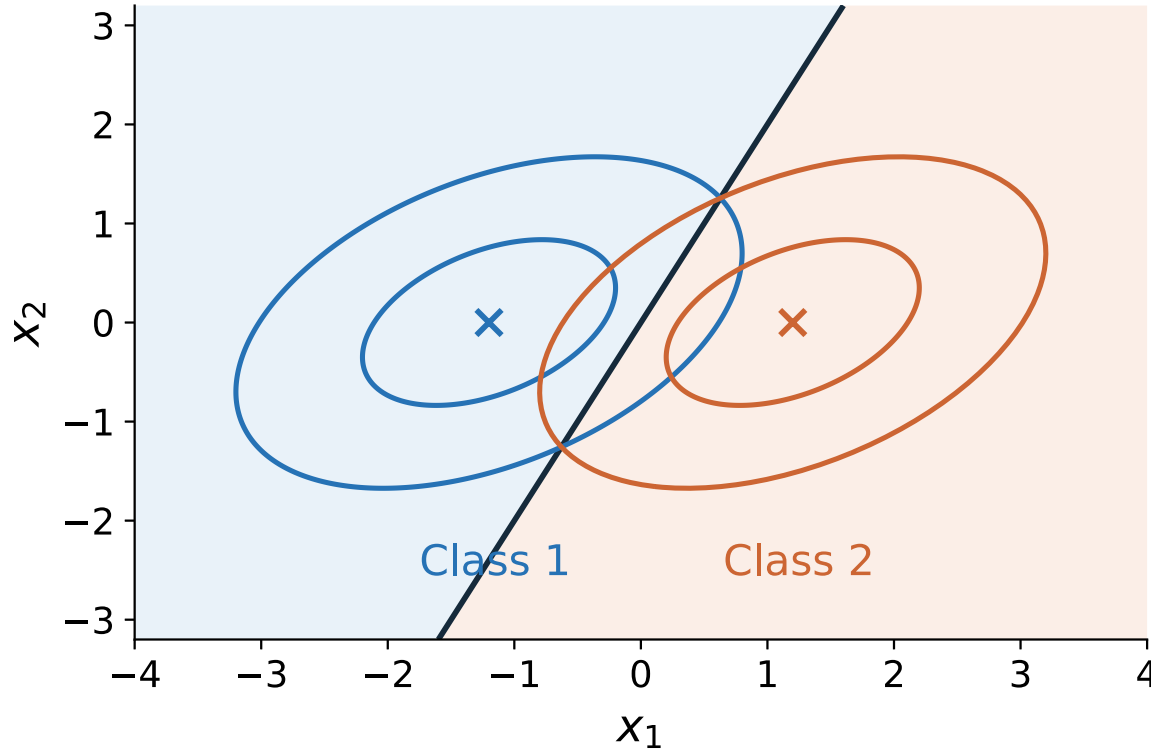
The first term and log determinant are shared; remove them from each score.

$$\delta_c(x) = x^T \Sigma^{-1} \mu_c - \frac{1}{2}\mu_c^T \Sigma^{-1} \mu_c + \log \pi_c$$

$$p_c(x) = \frac{e^{\delta_c(x)}}{\sum_{r=1}^C e^{\delta_r(x)}}$$

Equal scores give linear boundaries; normalized exponentials give softmax probabilities.

LDA produces the same linear log-odds form



For classes 1 and 2, subtract the scores:

$$\log \frac{p_1(x)}{p_2(x)} = \delta_1(x) - \delta_2(x)$$

$$= a + b^T x$$

$$b = \Sigma^{-1}(\mu_1 - \mu_2)$$

$$a = \log \frac{\pi_1}{\pi_2} - \frac{1}{2}(\mu_1^T \Sigma^{-1} \mu_1 - \mu_2^T \Sigma^{-1} \mu_2)$$

$$p_1(x) = \sigma(a + b^T x)$$

Shared covariance makes the quadratic terms cancel. The figure uses equal priors.
Does the same probability form imply the same fitted model?

Same probability form, different fitting objectives

Logistic regression maximizes the conditional log-likelihood:

$$\ell_{\text{cond}}(\beta) = \sum_i \log P_{\beta}(y_i | x_i)$$

Gaussian generative maximum likelihood fits the joint distribution.

$$\begin{aligned} \ell_{\text{joint}}(\theta) &= \sum_i \log [\pi_{y_i} f_{y_i}(x_i)] \\ &= \sum_i \log P_{\theta}(y_i | x_i) + \sum_i \log f_{\theta}(x_i) \\ f_{\theta}(x) &= \sum_c \pi_c f_c(x) \end{aligned}$$

θ contains the Gaussian parameters and priors. The extra marginal-density term generally changes the optimum, so fitted coefficients can differ.

Estimating LDA - and the next modeling choice

Let n_c count the training observations in class c . Estimate its prior and mean:

$$\hat{\pi}_c = \frac{n_c}{n}, \quad \hat{\mu}_c = \frac{1}{n_c} \sum_{i: y_i = c} x_i$$

Pool within-class deviations to estimate one shared covariance matrix:

$$\hat{\Sigma}_{\text{ML}} = \frac{1}{n} \sum_{c=1}^C \sum_{i: y_i = c} (x_i - \hat{\mu}_c)(x_i - \hat{\mu}_c)^T$$

Insert these estimates into the LDA scores to classify a new input.

What if the classes have different spreads or orientations?

A common unbiased pooled covariance divides by $n - C$ instead. Use a consistent convention.

Quadratic discriminant analysis (QDA)

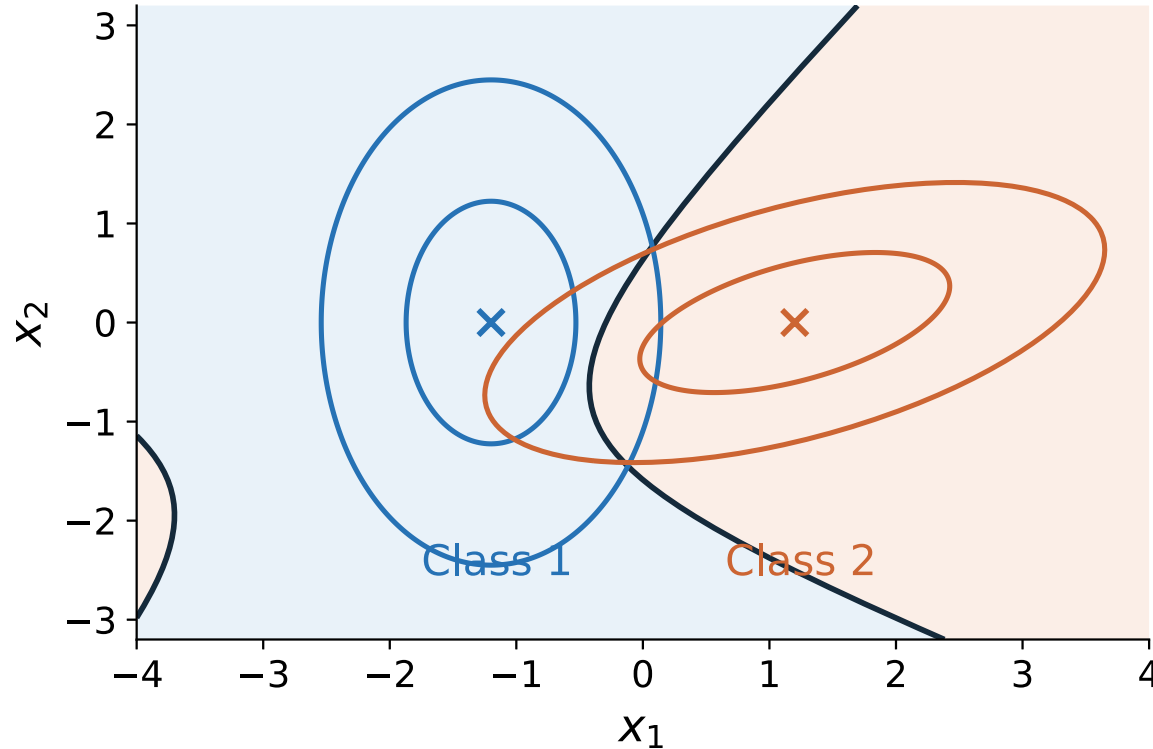
Allow Σ_a and Σ_b to differ. Subtract their Gaussian log scores.

$$\begin{aligned}\delta_a(x) - \delta_b(x) = & -\frac{1}{2}x^T(\Sigma_a^{-1} - \Sigma_b^{-1})x \\ & + x^T(\Sigma_a^{-1}\mu_a - \Sigma_b^{-1}\mu_b) + b_{ab}\end{aligned}$$

$$\begin{aligned}b_{ab} = & \log \frac{\pi_a}{\pi_b} - \frac{1}{2} \log \frac{|\Sigma_a|}{|\Sigma_b|} \\ & - \frac{1}{2}(\mu_a^T \Sigma_a^{-1} \mu_a - \mu_b^T \Sigma_b^{-1} \mu_b)\end{aligned}$$

Set this difference to zero. If $\Sigma_a = \Sigma_b$, the quadratic term vanishes and the LDA form returns.

QDA probabilities have a quadratic logistic form



For two classes, the previous expansion gives

$$\log \frac{p_1(x)}{p_2(x)} = a + b^T x + x^T A x$$

With two predictors and symmetric A:

$$x^T A x = A_{11}x_1^2 + 2A_{12}x_1x_2 + A_{22}x_2^2$$

$$\phi(x) = (1, x_1, x_2, x_1^2, x_1x_2, x_2^2)^T$$

$$p_1(x) = \sigma(\gamma^T \phi(x))$$

The figure uses unequal covariances and equal priors. QDA obtains the coefficients from Gaussian estimates; directly fitting logistic regression to these features generally gives different coefficients.

QDA needs more covariance estimates

Estimate a separate covariance from the observations in each class:

$$\hat{\Sigma}_{c, \text{ML}} = \frac{1}{n_c} \sum_{i: y_i = c} (x_i - \hat{\mu}_c)(x_i - \hat{\mu}_c)^T$$

A symmetric p by p matrix has $p(p + 1)/2$ distinct entries.

$$\text{LDA : } \frac{p(p + 1)}{2} \qquad \text{QDA : } C \frac{p(p + 1)}{2}$$

More flexibility requires more data in each class to estimate its spread reliably.

Does the extra flexibility improve predictions on held-out data?

Covariance counts only. An unbiased estimate uses one less than the class sample size. Invertibility requires more than p observations per class and full rank.

Smarket: an evaluation split that respects time

Response: Direction, with Up as the positive class.

Inputs: recent daily returns (Lag1 to Lag5) and Volume.

Data period	Role	Observations
2001-2004	Fit the model	998
2005	Evaluate future-period predictions	252

Today already determines Direction, so using it as an input would leak the answer.

Choose features, scaling and tuning using training data. This chronological split alone does not imply independence.

A confusion matrix identifies the kinds of errors

QDA on the 252 test observations, using Lag1 and Lag2.

Actual class	Predicted Down	Predicted Up
Down	30 (true negative)	81 (false positive)
Up	20 (false negative)	121 (true positive)

$$\text{Accuracy} = \frac{30 + 121}{252} = 59.9\%$$

Rows are actual classes and columns are predictions. The positive class is Up.

Different error costs change the decision threshold

Let $p(x)$ be the probability of class 1. Correct decisions have zero cost.

A false positive and a false negative may have different positive costs:

$$R(1 \mid x) = C_{\text{FP}}[1 - p(x)], \quad R(0 \mid x) = C_{\text{FN}}p(x)$$

Predict class 1 when its expected cost is lower:

$$C_{\text{FP}}[1 - p(x)] < C_{\text{FN}}p(x) \implies p(x) > \frac{C_{\text{FP}}}{C_{\text{FP}} + C_{\text{FN}}}$$

Equal costs give a threshold of 0.5. Higher false-negative cost lowers it.

The threshold is a decision choice. We next examine how changing it affects the error rates.

Thresholds trade false positives for false negatives

Predict Up if its estimated probability exceeds t .

Lowering t weakly increases both the true-positive rate and the false-positive rate.

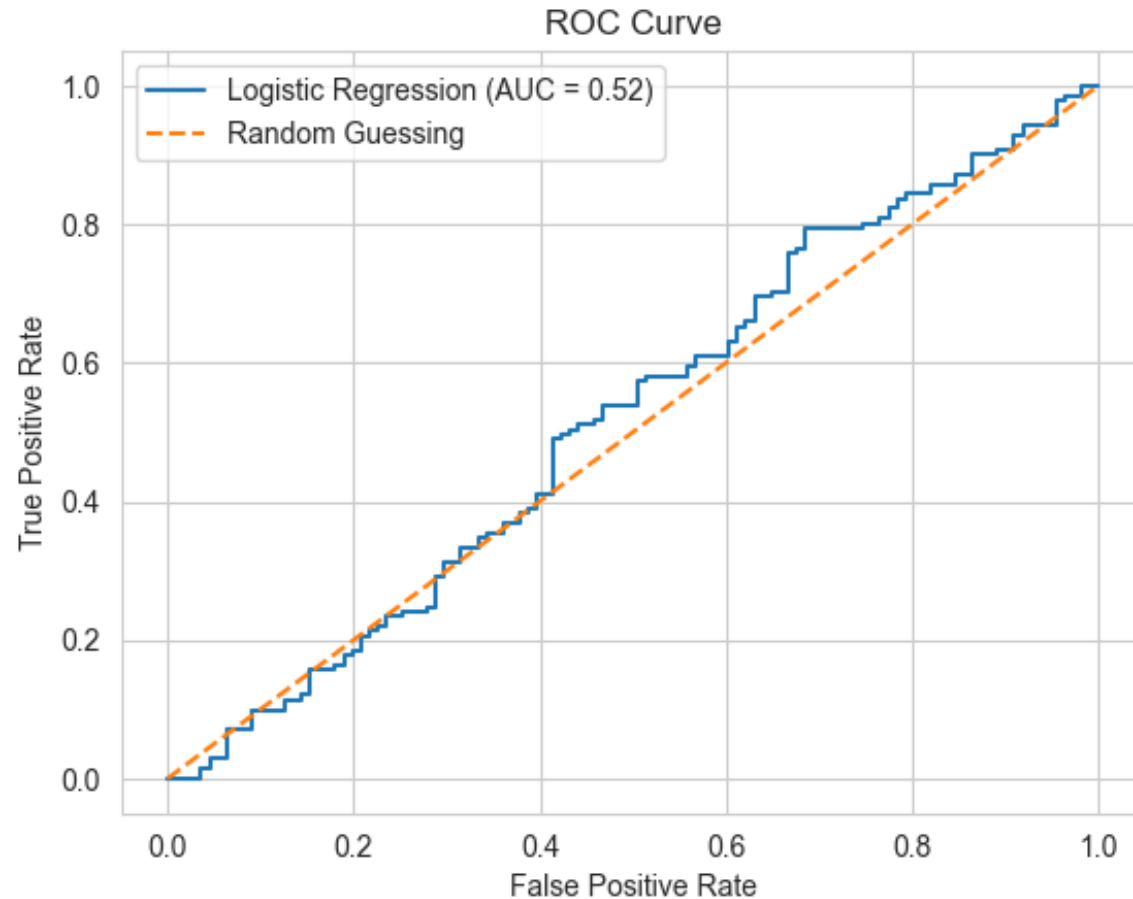
$$\text{TPR} = \frac{\text{TP}}{\text{TP} + \text{FN}}, \quad \text{FPR} = \frac{\text{FP}}{\text{FP} + \text{TN}}$$

$$\text{Precision} = \frac{\text{TP}}{\text{TP} + \text{FP}}$$

Recall is another name for TPR. Precision asks how often a positive prediction is correct.

Choose a threshold using the error costs or a validation objective.

ROC and AUC evaluate ranking across thresholds



Each ROC point uses one threshold:

horizontal axis: FPR

vertical axis: TPR

AUC is the area under the ROC curve. It measures ranking quality.

AUC = 0.5 corresponds to chance-level ranking. Calibration is a separate property.

Smarket, six-input logistic model on 2005: AUC $\approx$ 0.52, accuracy at threshold 0.5 $\approx$ 48.0%.

Smarket results on the same 2005 test set

Classifier	Inputs	Correct / 252	Accuracy
Always predict Up	None	141	56.0%
Logistic regression	Lag1-Lag5, Volume	121	48.0%
Logistic regression	Lag1, Lag2	141	56.0%
LDA	Lag1, Lag2	141	56.0%
QDA	Lag1, Lag2	151	59.9%
1-nearest neighbor	Lag1, Lag2	126	50.0%

One held-out period gives limited evidence. Repeatedly choosing models on this test set makes it part of tuning.

Caravan: class imbalance changes the comparison

5,822 individuals, 85 predictors, response Purchase (Yes / No).

In the first 1,000 observations, 59 bought insurance.

$$\text{Accuracy(always No)} = \frac{941}{1000} = 94.1\%$$

For a KNN exercise:

1. Hold out the first 1,000 cases; use the remaining 4,822 for training.
2. Estimate scaling within training data and select k by validation.
3. Report precision and recall for Yes alongside accuracy.

A high majority-class accuracy can coexist with recall equal to zero for buyers.

Appendix: further mathematical details

Supporting derivations for reference

Distances and transformations	34
Logistic uncertainty and existence of the MLE	35
Multiclass logistic regression: full derivations	36
Deriving Gaussian parameter estimates	39
LDA as a distance to class centers	40
Multiclass QDA and quadratic softmax scores	41

Select a topic to jump to the corresponding derivation.

Distances, scaling and linear transformations

For positive definite M , choose an invertible matrix A with $M = A^T A$.

$$\begin{aligned} d_M(x, x')^2 &= (x - x')^T A^T A (x - x') \\ &= \|A(x - x')\|_2^2 \end{aligned}$$

Thus a Mahalanobis-type distance is Euclidean distance after transforming the features.

$$M = \text{diag}(s_1^{-2}, \dots, s_p^{-2})$$

This choice gives distance after standardization. $M = \Sigma^{-1}$ also accounts for correlations when Σ is invertible.

Logistic uncertainty and existence of the MLE

Under the model and regularity conditions, information comes from likelihood curvature.

$$\mathcal{I}(\beta) = X^T W X$$

$$\widehat{\text{Cov}}(\hat{\beta}) \approx (X^T \widehat{W} X)^{-1}$$

$$\frac{\hat{\beta}_j - \beta_j}{\widehat{\text{SE}}(\hat{\beta}_j)} \overset{\text{approx}}{\sim} \mathcal{N}(0, 1)$$

Perfect class separation can drive coefficients toward infinity, so an unpenalized finite MLE may not exist.

These are large-sample normal approximations. The linear-regression exact t argument does not apply.

Baseline log odds imply softmax probabilities

Assume $C - 1$ linear log odds relative to class C .

$$\log \frac{p_c}{p_C} = \eta_c \quad \implies \quad p_c = p_C e^{\eta_c}$$

$$1 = \sum_{c=1}^C p_c = p_C \left(1 + \sum_{c=1}^{C-1} e^{\eta_c} \right)$$
$$p_C = \frac{1}{1 + \sum_{r < C} e^{\eta_r}}, \quad p_c = \frac{e^{\eta_c}}{1 + \sum_{r < C} e^{\eta_r}}$$

Adding the same score to every class leaves softmax unchanged. Setting the baseline score to zero removes this ambiguity.

Multiclass likelihood and its derivatives

Let $y_{ic} = 1$ if observation i belongs to class c , and 0 otherwise.

$$\begin{aligned}\ell(\beta_1, \dots, \beta_{C-1}) &= \sum_i \sum_c y_{ic} \log p_{ic} \\ &= \sum_i \left[\sum_{c < C} y_{ic} \beta_c^T \tilde{x}_i - \log \left(1 + \sum_{c < C} e^{\beta_c^T \tilde{x}_i} \right) \right]\end{aligned}$$

$$\frac{\partial \ell}{\partial \beta_c} = \sum_i \tilde{x}_i (y_{ic} - p_{ic})$$

$$\frac{\partial^2 \ell}{\partial \beta_c \partial \beta_d^T} = - \sum_i p_{ic} (\mathbb{I}(c = d) - p_{id}) \tilde{x}_i \tilde{x}_i^T$$

Derivatives use $c, d < C$. The shared denominator couples the classes, so solve jointly by numerical optimization.

Pairwise boundaries and the binary special case

$$\frac{p_a}{p_b} = \frac{e^{\eta_a}}{e^{\eta_b}} \implies \log \frac{p_a}{p_b} = (\beta_a - \beta_b)^T \tilde{\mathbf{x}}$$

Classes a and b tie when their score difference is zero.

$$(\beta_a - \beta_b)^T \tilde{\mathbf{x}} = 0$$

This equality is an active classification boundary only where their shared score is maximal.

$$C = 2 : \quad p_1 = \frac{e^{\eta_1}}{1 + e^{\eta_1}} = \sigma(\eta_1)$$

Deriving the Gaussian mean and covariance estimates

Differentiating the Gaussian log-likelihood with respect to the class mean:

$$0 = \sum_{i: y_i = c} \Sigma_c^{-1} (x_i - \mu_c) \implies \hat{\mu}_c = \frac{1}{n_c} \sum_{i: y_i = c} x_i$$

Let R be the sum of m centered outer products, and S the covariance.

$$\ell(S) = \text{constant} - \frac{m}{2} \log |S| - \frac{1}{2} \text{tr}(S^{-1}R)$$

$$0 = -mS^{-1} + S^{-1}RS^{-1} \implies \hat{S} = \frac{R}{m}$$

Here tr denotes the trace. LDA pools all classes with $m = n$. QDA uses one class with $m = n_c$. The prior MLE is n_c / n .

LDA, Mahalanobis distance and class centers

Write the LDA score using the squared Mahalanobis distance:

$$d_c^2(x) = (x - \mu_c)^T \Sigma^{-1} (x - \mu_c)$$

$$\delta_c(x) = -\frac{1}{2}d_c^2(x) + \log \pi_c + \frac{1}{2}x^T \Sigma^{-1} x$$

The last term is the same for every class. Therefore:

$$\arg \max_c \delta_c(x) = \arg \min_c [d_c^2(x) - 2\log \pi_c]$$

$$R^T R = \Sigma^{-1} \implies d_c^2(x) = \|Rx - R\mu_c\|_2^2$$

Equal priors give nearest class means after a common transformation. KNN instead uses distances to training cases. QDA also needs class-specific log determinants.

Multiclass QDA uses quadratic softmax scores

For each QDA class, expand the log prior plus the Gaussian log density:

$$s_c(x) = -\frac{1}{2}x^T \Sigma_c^{-1} x + x^T \Sigma_c^{-1} \mu_c + a_c$$

$$a_c = \log \pi_c - \frac{1}{2} \log |\Sigma_c| - \frac{1}{2} \mu_c^T \Sigma_c^{-1} \mu_c$$

The Gaussian normalization term common to every class cancels.

$$p_c(x) = \frac{e^{s_c(x)}}{\sum_{r=1}^C e^{s_r(x)}}$$

Quadratic softmax scores can therefore express QDA probabilities.

QDA estimates Gaussian parameters first. Direct softmax training fits its score coefficients by conditional likelihood.